

MACHINE LEARNING INTERNSHIP PROGRAM

**AI ADMISSION PREDICTOR:
COLLEGE ADMISSION PREDICTION USING MACHINE
LEARNING**

AN INTERNSHIP PROJECT REPORT

Submitted by

J. SAM JACOB

(Reg No:110725102213)

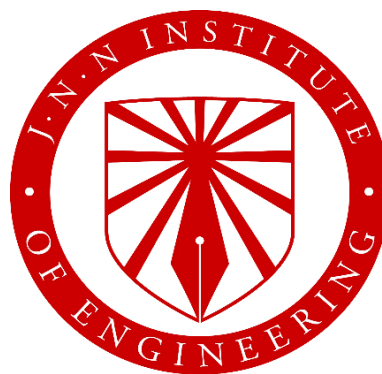
in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Data Science



J.N.N Institute of Engineering

(An Autonomous Institute , Affiliated to Anna University, Chennai)

OCTOBER – 2026



VISION OF THE INSTITUTE:

Lead the transformation of engineering and management learning experience to educate the next generation of innovators and entrepreneurs who want to make the world a better place.

MISSION OF THE INSTITUTE:

IM1: To develop the required resources and infrastructure and to establish a conducive ambience for the teaching-learning process.

IM2: To nurture professional and ethical values in the students and to instils in them a spirit of innovation and entrepreneurship.

IM3: To encourage a desire for higher learning and research in the students and to equip them to face global challenges.

IM4: To provide opportunities for students to learn job-relevant skills to make them industry ready.

IM5: To interact with industries and other organization to facilitate transfer of knowledge and know-how.



VISION OF THE DEPARTMENT:

To impart quality-education, inculcate professionalism and enhance the problem-solving skills of the students in the domain of Artificial Intelligence & Data Science by applying recent technological tools and incorporating collaborative principles with a focus to make them industry ready.

MISSION OF THE DEPARTMENT:

- **DM 1:** To enhance the knowledge of the students with most recent advancements and refresh their insights in the field of Artificial Intelligence and Data Science.
- **DM 2:** To equip the students with strong fundamental concepts, analytical capability, programming and problem-solving skills.
- **DM 3:** To make the students industry ready and to enhance their employability through training, internships and real-time projects.
- **DM 4:** To guide the students to perform research on Artificial Intelligence and Data Science, with the aim to provide solutions to the problems of the industry



PROGRAM OUTCOMES

S.NO	GRADUATE ATTRIBUTE	PROGRAM OUTCOMES
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The Engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
PO7	Environment and sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.



Department of Artificial Intelligence & Data Science

PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

S.No	PROGRAM EDUCATIONAL OBJECTIVES (PEO)
PEO1	To provide graduates with the proficiency to utilize the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volume of data.
PEO2	To enrich graduates with necessary technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
PEO3	To enable graduates to think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**

NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



Department of Artificial Intelligence & Data Science

PROGRAM SPECIFIC OBJECTIVES (PSOs)

S.No	PROGRAM SPECIFIC OBJECTIVES (PSOs)
PSO1	Graduates should be able to evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.
PSO2	Graduates should be able to arrive at actionable Fore sight, Insight , hind sight from data for solving business and engineering problems
PSO3	Graduates should be able to create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems.



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**
NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



BONAFIDE CERTIFICATE

Certified that this internship project report
“**AI/ML ADMISSION PREDICTOR MODEL – COLLEGE ADMISSION PREDICTION
USING MACHINE LEARNING**”
(Machine Learning Internship) is the bonafide work of “**J. SAM JACOB**”
(Reg No: 110725102213) who carried out the project under my supervision.

Signature of the HOD with date

Dr. P. Nagarajan

Professor and Head,
Department of Artificial Intelligence
& Data Science Engineering,
**J.N.N Institute of Engineering,
Chennai**

Signature of the Supervisor with date

Miss. Brindha . S

Project Supervisor,
Department of Artificial Intelligence
& Data Science Engineering,
**J.N.N Institute of Engineering,
Chennai**

Certified that the above students have attend of viva voce during the exam
held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to our Beloved **FOUNDER & CHAIRMAN Shri. S. JAYACHANDRAN B.Sc., B.L.**, and also heartfelt thanks to our **VICE CHAIRMAN Mr. NAVEEN JAYAHANDRAN M.S.**, for providing necessary facilities for the completion of our project.

We thank our **PRINCIPAL Dr. G. MOHANBABU, Ph.D.**, for the encouragement provided with the congenial environment essential for us to complete the project.

We take this opportunity to express our gratitude to our **HEAD OF THE DEPARTMENT Dr. P. NAGARAJAN** and all other faculty members in the Department of ARTIFICIAL INTELLIGENCE AND DATA SCIENCE for giving us continuous encouragement for completing our project.

We extend our sincere thanks to **KanmalaiTech UK Limited** and its **Director, Mr. Ramesh Balakrishnan**, for giving us the opportunity to undergo this Machine Learning internship and for his valuable guidance. We also sincerely thank **the entire KanmalaiTech team** for their guidance, patience and technical support throughout this internship.

Finally, we would like to thank our parents and friends for their constant support and encouragement.

ABSTRACT

Colleges must plan seats, staff and facilities long before the academic year begins, and this depends on how many students are likely to join. Machine learning allows this estimate to be made from data instead of guesswork. This report documents the work carried out during the Machine Learning Programme at KanmalaiTech UK Limited, completed online between 3rd August 2026 and 28th August 2026 under the guidance of Mr. Ramesh Balakrishnan, Director.

The internship moved through eight connected activities. The intern learned the fundamentals of machine learning, practised Python with NumPy and pandas, trained and compared Linear Regression and Random Forest models, and then developed the AI Admission Predictor, a web application that accepts seven factors (applications received, seats available, placement percentage, advertisement budget, courses available, annual fees and last year's admissions) and displays the predicted number of admissions with a bar chart and a seat-utilisation chart. The application was deployed on Render, the code was maintained on GitHub and a project poster was prepared.

The work shows that a student-level machine learning project can give useful decision support for admission planning when the data is handled carefully and the result is clearly presented. The internship gave the intern a deployed project, practical skills in Python and scikit-learn, and exposure to a professional machine learning workflow.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
1	INTRODUCTION	1
	1.1 OVERVIEW	1
	1.2 INTERNSHIP DETAILS	1
	1.3 OBJECTIVES	2
	1.4 OVERVIEW OF THE INTERNSHIP ACTIVITIES	2
	1.5 PROBLEM STATEMENT	3
	1.6 SCOPE OF THE WORK	3
	1.7 ORGANISATION OF THE REPORT	3
2	LITERATURE REVIEW	4
	2.1 MACHINE LEARNING	4
	2.2 SUPERVISED LEARNING AND REGRESSION	4
	2.3 FEATURES, LABELS AND DATASETS	4
	2.4 MODEL EVALUATION	4
	2.5 PYTHON AND ITS DATA LIBRARIES	5
	2.6 WEB APPLICATIONS AND APIS	5
	2.7 DATA VISUALISATION	5
	2.8 CLOUD DEPLOYMENT AND VERSION CONTROL	5
	2.9 TOOLS AND THEIR PURPOSE	5
	2.10 EXISTING APPROACHES AND THEIR LIMITATIONS	6
	2.11 NEED FOR THE APPROACH FOLLOWED	6
3	METHODOLOGY	7
	3.1 OVERALL APPROACH	7
	3.2 INTERNSHIP SCHEDULE	7
	3.3 SYSTEM ARCHITECTURE	7
	3.4 INPUT FACTORS	8
	3.5 MODEL DEVELOPMENT METHODOLOGY	8
	3.6 WEB APPLICATION AND DEPLOYMENT METHODOLOGY	8
	3.7 SYSTEM REQUIREMENTS	9
	3.8 TESTING METHOD	9
	3.9 DATA AND ETHICAL CONSIDERATIONS	9
4	IMPLEMENTATION	10
	4.1 MODULE 1 – MACHINE LEARNING FUNDAMENTALS AND PYTHON SETUP	10
	4.2 MODULE 2 – DATA HANDLING WITH PANDAS	11
	4.3 MODULE 3 – MODEL TRAINING AND EVALUATION	12
	4.4 MODULE 4 – INPUT INTERFACE OF THE AI ADMISSION PREDICTOR	12
	4.5 MODULE 5 – PREDICTION SERVICE	13

CHAPTER NO	TITLE	PAGE NO.
	4.6 MODULE 6 – RESULT VISUALISATION	14
	4.7 MODULE 7 – DEPLOYMENT AND VERSION CONTROL	15
	4.8 MODULE 8 – PROJECT POSTER	16
	4.9 TESTING	16
	4.10 CHALLENGES AND SOLUTIONS	17
5	RESULTS AND DISCUSSION	18
	5.1 RESULTS OBTAINED	18
	5.2 FEATURES OF THE AI ADMISSION PREDICTOR	18
	5.3 SKILLS GAINED	19
	5.4 COMPARISON WITH MANUAL ESTIMATION	19
	5.5 INTERNSHIP EXPERIENCE	19
	5.6 DISCUSSION	19
	5.7 RESULTS OF THE PRACTICE PROGRAMS	20
6	CONCLUSION AND FUTURE WORK	21
	6.1 CONCLUSIONS	21
	6.2 FUTURE WORK	21
	6.3 SKILLS DEVELOPED	21
	6.4 LIMITATIONS	21
	6.5 BENEFITS OF THE INTERNSHIP	21
	REFERENCES	22
	APPENDIX – INTERNSHIP CERTIFICATE	23

CHAPTER 1

INTRODUCTION

1.1 Overview

Every college must decide, long before the academic year begins, how many students it is likely to admit. The decision affects the number of sections to be opened, the number of faculty members to be appointed, the classrooms, laboratories and hostel rooms to be kept ready, and the money to be spent on advertising. When this estimate is made only by experience and guesswork, the college may be left with empty seats or, on the other hand, with more students than its facilities can handle.

Machine Learning (ML) offers a data-driven way to make this estimate. An ML model studies the records of earlier years – applications received, seats available, placement percentage, advertisement spending, courses offered, fees and the previous year's admissions – and learns how these factors together influence the number of students who finally join. Once trained, the model can estimate the admissions for a new year within a moment.

This report presents the work carried out during the Machine Learning Programme at KanmalaiTech UK Limited. The intern began with the basic ideas of machine learning and Python, practised data handling and model building through small programs, and finally developed a working web application, the AI Admission Predictor, which accepts seven admission-related factors and displays the predicted number of admissions along with a bar chart and a seat-utilisation chart.

1.2 Internship Details

Table 1.1 – Details of the internship

Particulars	Details
Name of the Intern	J. SAM JACOB
Register Number	110725102213
Department	Artificial Intelligence & Data Science
Year / Semester	II Year / III Semester (Batch 2025 – 2029)
Institution	J.N.N Institute of Engineering, Chennai
Organisation	KanmalaiTech UK Limited
Programme / Role	Machine Learning Programme – Machine Learning Intern
Internship Period	3rd August 2026 to 28th August 2026 (4 weeks)
Mode	Online (Remote)
Guide	Mr. Ramesh Balakrishnan, Director, KanmalaiTech UK Ltd
Project Undertaken	AI Admission Predictor – College Admission Prediction using Machine Learning
Live Project Link	https://admission-predictor-kazl.onrender.com
Certificate	Certificate of Successful Completion, issued on 01-Sep-2026

1.3 Objectives

- To understand the basic concepts of machine learning, including supervised learning, features, labels, training and testing.
- To learn Python programming with the NumPy and pandas libraries for handling and exploring data.
- To train regression models using scikit-learn and to evaluate them with suitable error measures.
- To build a college admission prediction application that takes seven input factors and returns the expected number of admissions.
- To present the prediction clearly through a web interface with a bar chart and a seat-utilisation chart.
- To deploy the application online and to maintain its source code on GitHub, and to prepare a project poster.

1.4 Overview of the Internship Activities

The internship was divided into the following connected activities. Each activity is described as a module in Chapter 4.

- Module 1 – Machine Learning Fundamentals and Python Setup: types of machine learning, the ML workflow and basic NumPy operations.
- Module 2 – Data Handling with pandas: creating, loading, inspecting and summarising an admissions dataset.
- Module 3 – Model Training and Evaluation: splitting data, training Linear Regression and Random Forest models and comparing their errors.
- Module 4 – Input Interface of the AI Admission Predictor: a web form with seven factors and their typical ranges.
- Module 5 – Prediction Service: back-end logic that validates the inputs and returns the predicted admissions.
- Module 6 – Result Visualisation: a bar chart of applications, seats and predicted admissions, and a seat-utilisation chart.
- Module 7 – Deployment and Version Control: hosting the application on Render and keeping the code on GitHub.
- Module 8 – Project Poster: an infographic that explains the model, the factors considered and its uses.



Fig. 1.1 – Phases of the internship work flow

1.5 Problem Statement

Colleges usually estimate their next-year admissions from last year's figure or from the opinion of the management. Such estimates do not take into account several factors that act together, for example a sudden rise in applications, a change in the advertisement budget or a better placement record. A wrong estimate leads to poor planning of seats, staff and infrastructure.

The internship therefore addressed a practical question: how can a student-level machine learning project turn a handful of admission-related factors into a quick, easy-to-read prediction that helps the management plan for the next academic year?

1.6 Scope of the Work

The work was carried out as a learning-oriented internship and is limited to the following boundaries.

- Included: learning the fundamentals of machine learning, Python, NumPy and pandas through small programs.
- Included: building an admission predictor that uses seven input factors and displays the predicted admissions with two charts.
- Included: hosting the application online, keeping the code on GitHub and preparing a project poster.
- Not included: integration with a real college database, user login, automatic yearly retraining and a large multi-college dataset, which are suggested as future work in Chapter 6.

1.7 Organisation of the Report

The report is organised into six chapters. Each chapter builds on the one before it.

- Chapter 1 – Introduction: states the purpose of the internship, the problem, the objectives, the scope and the activities.
- Chapter 2 – Literature Review: explains machine learning, regression, model evaluation, the tools used and the limits of existing approaches.
- Chapter 3 – Methodology: describes the approach, the weekly schedule, the system architecture, the requirements and the testing method.
- Chapter 4 – Implementation: presents each module with its code, output and screenshots, followed by testing and the difficulties faced.
- Chapter 5 – Results and Discussion: discusses the results of the predictor, its features, the skills gained and a comparison with manual estimation.
- Chapter 6 – Conclusion and Future Work: summarises the work, the limitations and the future scope.

CHAPTER 2

LITERATURE REVIEW

This chapter gives the background of the concepts, techniques and tools used during the internship.

2.1 Machine Learning

Machine learning is the branch of artificial intelligence in which a computer improves its performance on a task by learning from data instead of following rules written by a programmer for every situation [1]. A learning program is given examples; it finds the pattern that links the inputs with the outputs and then applies that pattern to new inputs it has never seen.

Machine learning problems are commonly grouped into supervised learning, unsupervised learning and reinforcement learning. In supervised learning every example carries the correct answer, and the model learns to reproduce it. In unsupervised learning there are no answers and the model looks for structure such as groups. In reinforcement learning an agent learns by receiving rewards for its actions [2]. The admission predictor built in this internship is a supervised learning problem.

2.2 Supervised Learning and Regression

Supervised learning tasks are of two kinds. Classification predicts a category, such as whether an e-mail is spam. Regression predicts a number, such as a house price or the number of students who will join a college. Because the output of the admission predictor is a count of students, it is a regression task.

Linear Regression fits a straight-line relationship between the inputs and the output and is easy to interpret. A Random Forest builds many decision trees on different samples of the data and averages their answers; it can capture curved and interacting relationships and is less affected by a single unusual record [13]. Both were tried during the internship.

2.3 Features, Labels and Datasets

The inputs of a model are called features and the answer it must predict is called the label. In the admission predictor the features are the applications received, seats available, placement percentage, advertisement budget, number of courses, annual fees and last year's admissions; the label is the number of admissions. A collection of such rows is the dataset.

The quality of a model depends on the quality of its data. Missing values, wrong entries and values far outside the normal range must be found and corrected before training. The features are also usually kept within sensible ranges, which is why the web form of the project shows a typical range below every field.

2.4 Model Evaluation

A model must be tested on data it has not seen during training. For this reason the dataset is divided into a training part and a testing part. Two common measures were used in the practice programs. The Mean Absolute Error (MAE) is the average size of the difference between the predicted and the

actual value; a smaller value is better. The coefficient of determination (R^2) tells how much of the variation in the output is explained by the model; a value nearer to 1 is better [3].

A model that scores very well on the training rows but badly on the testing rows has memorised the data instead of learning the pattern; this is called overfitting. Keeping a separate testing part, and comparing more than one model, is the simplest protection against it. This is why the internship compared Linear Regression with Random Forest on the same testing rows.

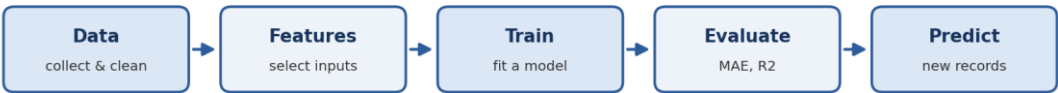


Fig. 2.1 – Typical machine learning pipeline

2.5 Python and its Data Libraries

Python is a high-level language whose readable syntax has made it the most popular language for machine learning [14], [15]. NumPy provides fast arrays and mathematical functions [7]. pandas provides the DataFrame, a table-like structure that makes loading, cleaning and summarising data simple [5], [6]. scikit-learn gives ready-made, consistent implementations of many ML algorithms together with tools for splitting data and measuring errors [3], [4].

2.6 Web Applications and APIs

A web application lets a user work with a program through a browser. The page that the user sees is the front end; the program that receives the data, runs the model and sends back the answer is the back end. The two communicate through an Application Programming Interface (API), commonly with data in JSON format. Flask is a lightweight Python framework that makes it easy to create such an API around a trained model [8].

2.7 Data Visualisation

A number alone is difficult to interpret. A chart shows its meaning at a glance: a bar chart compares quantities side by side and a doughnut chart shows how a whole is divided into parts. JavaScript libraries such as Chart.js draw such charts in the browser from a few lines of configuration [9].

2.8 Cloud Deployment and Version Control

A project becomes useful only when other people can open it. Cloud platforms such as Render build an application from a code repository and publish it on a public web address [10]. Git records every change made to the source code and GitHub stores the repository online, so that the work is safe and can be shared [11], [12].

2.9 Tools and Their Purpose

Table 2.1 – Tools and technologies used

Tool / Technology	Purpose in the internship
Python	Main programming language for data handling, model training and the back end

Tool / Technology	Purpose in the internship
NumPy	Array calculations such as mean, standard deviation and scaling
pandas	Loading, inspecting and summarising the admissions dataset
scikit-learn	Splitting data, training Linear Regression and Random Forest models, and measuring MAE and R^2
Flask	Receiving the form data and returning the prediction as JSON
HTML, CSS, JavaScript	Building the input form and the result page
Chart.js	Drawing the bar chart and the seat-utilisation chart
Git and GitHub	Version control and online storage of the project code
Render	Cloud hosting of the live application

2.10 Existing Approaches and Their Limitations

Colleges generally estimate admissions in one of three ways.

- Last-year comparison: the previous year's figure is simply repeated. It is quick, but it ignores changes in applications, fees, advertising or placements.
- Management judgement: experienced people give an opinion. It is useful, but it is subjective and cannot be repeated or explained with numbers.
- Spreadsheet calculation: fixed percentages are applied to the number of applications. It is easy to follow, but a fixed percentage cannot capture the combined effect of several factors.
- Large analytics software: professional tools exist, but they are costly and need trained analysts, which is beyond the reach of most small institutions.

2.11 Need for the Approach Followed

The approach followed in the internship overcomes these limits. A trained regression model weighs all seven factors together, gives the same answer for the same inputs, and can be updated when new data becomes available. By wrapping the model in a simple web page with charts, a non-technical user can obtain and understand the prediction without writing any code, and the free cloud hosting makes it available to any browser.

CHAPTER 3 METHODOLOGY

3.1 Overall Approach

The internship followed a step-by-step approach in which each stage prepared the ground for the next. The intern first learned the concepts, then practised them in small Python programs, prepared a dataset, trained and compared models, and finally built, tested and deployed the web application. Figure 3.1 shows the sequence.

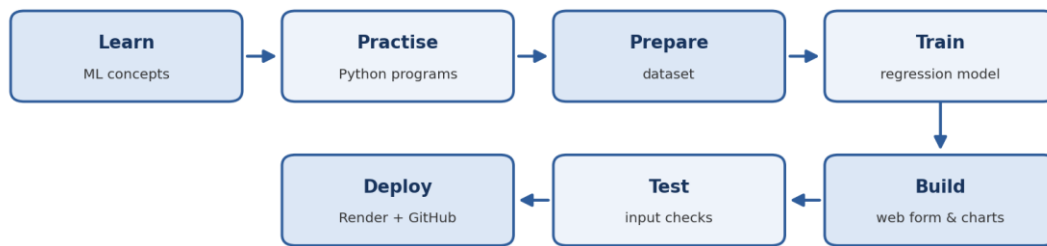


Fig. 3.1 – Overall methodology of the internship

3.2 Internship Schedule

The internship ran for four weeks in online mode. Table 3.1 summarises the activities of each week.

Table 3.1 – Week-wise activities during the internship

Period	Activities
Week 1 3 – 7 August	Introduction to machine learning; types of learning; the ML workflow; Python environment set-up; NumPy basics (Module 1)
Week 2 10 – 14 August	pandas DataFrames; loading and summarising data; preparing the admissions dataset (Module 2)
Week 3 17 – 21 August	Train-test split; Linear Regression and Random Forest; MAE and R ² ; choosing the model (Module 3)
Week 4 24 – 28 August	Admission Predictor web interface, prediction service, charts, testing, deployment on Render, GitHub upload and project poster (Modules 4 – 8)

3.3 System Architecture

The AI Admission Predictor has a simple three-part architecture. The user enters seven values in a browser form. The form data is sent to the back end, which validates it and passes it to the trained model. The predicted number is returned to the browser, where it is shown together with two charts.

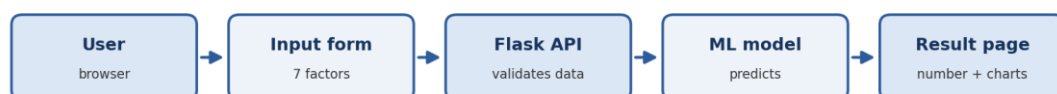


Fig. 3.2 – Architecture of the AI Admission Predictor

3.4 Input Factors

The model uses seven factors, which are also the seven fields of the web form. Table 3.2 lists them with the typical range shown beside each field in the application.

Table 3.2 – Input factors of the AI Admission Predictor

Factor	Meaning	Typical range
Applications Received	Number of applications received for the year	1000 – 3000
Seats Available	Total sanctioned seats	800 – 2200
Placements (%)	Placement percentage of the previous batch	55 – 98
Advertisement Budget (₹)	Money spent on promotion	₹80,000 – ₹8,00,000
Courses Available	Number of courses offered	5 – 12
Annual Fees (₹)	Fees charged per year	₹75,000 – ₹2,50,000
Last Year Admissions	Students admitted in the previous year	Similar to the seats range

3.5 Model Development Methodology

- Collect: assemble the records of the seven factors and the number of admissions.
- Clean: check for missing and unreasonable values.
- Split: keep 80 per cent of the rows for training and 20 per cent for testing.
- Train: fit Linear Regression and Random Forest models on the training rows.
- Evaluate: compare the models using MAE and R^2 on the testing rows.
- Select: use the better model for the predictor and keep it ready for the back end.

To learn this method safely, the practice programs of Chapter 4 use a small practice dataset generated by the program itself, so that every step can be repeated and its output checked.

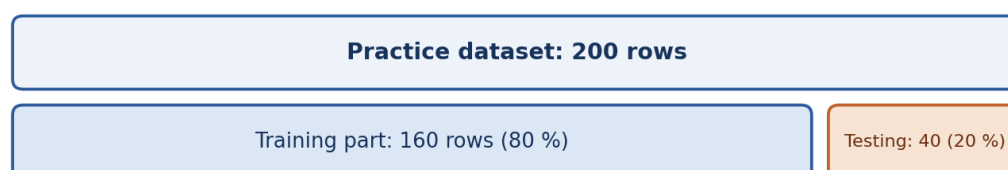


Fig. 3.3 – Train-test split used in the practice programs

3.6 Web Application and Deployment Methodology

The input form and the result page were designed with HTML, CSS and JavaScript in a dark theme with clear labels. The back end validates every field before the model is called, so that incomplete data never reaches the model. The charts are drawn after the prediction arrives. The finished project was pushed to GitHub and then connected to Render, which builds it and publishes it at a public address.

3.7 System Requirements

Table 3.3 – Hardware and software requirements

Item	Requirement
Computer	Any laptop or desktop with 4 GB RAM or more
Operating system	Windows, Linux or macOS
Programming language	Python 3
Libraries	NumPy, pandas, scikit-learn, Flask
Browser	Any modern browser such as Chrome or Edge
Online services	GitHub account and Render account
Internet	Required for online sessions, GitHub and Render

3.8 Testing Method

The work was tested at two levels. The practice programs were run and their output was compared with what was expected by hand. The web application was tested by entering sample values, pressing the Predict Admissions button, and checking that the prediction, the bar chart and the seat-utilisation chart appeared correctly. Missing fields were tested to confirm that the back end returns a clear error message instead of a wrong answer.

3.9 Data and Ethical Considerations

A prediction about admissions concerns real students, so it must be used with care. The model should be trained only on genuine, lawfully collected institutional data and must not contain personal details of applicants; the seven factors of this project are all institution-level figures. The output is meant to help the management plan seats, staff and facilities. It must not be used to reject or select individual applicants, and its limits must be explained to anyone who uses it.

CHAPTER 4

IMPLEMENTATION

This chapter describes the practical execution of each module, including the code, its output and the screenshots of the project. The programs of Modules 1 to 3 and Module 5 are practice and representative programs written to explain each step; they use a practice dataset created by the program itself. The screenshots of Modules 4, 6 and 8 are taken from the live AI Admission Predictor.

4.1 Module 1 – Machine Learning Fundamentals and Python Setup

Services used: Python 3, NumPy.

The first week introduced the meaning of machine learning, the three kinds of learning and the standard ML workflow. Python was installed and NumPy was used to understand arrays, because every ML library stores its data in arrays. The following program computes the mean, the standard deviation, a demand ratio (applications per seat) and a min-max scaled version of five sample values.

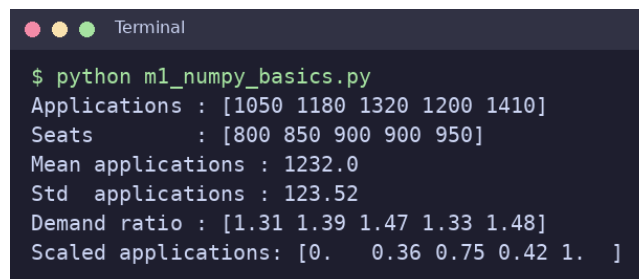
```
import numpy as np

# Applications received in five sample years
applications = np.array([1050, 1180, 1320, 1200, 1410])
seats = np.array([800, 850, 900, 900, 950])

print("Applications :", applications)
print("Seats          :", seats)
print("Mean applications :", applications.mean())
print("Std applications :", round(applications.std(), 2))

# Applications per seat (a simple demand ratio)
demand_ratio = applications / seats
print("Demand ratio :", np.round(demand_ratio, 2))

# Min-max scaling brings values into the range 0 to 1
lo, hi = applications.min(), applications.max()
scaled = (applications - lo) / (hi - lo)
print("Scaled applications:", np.round(scaled, 2))
```

A terminal window titled "Terminal" with a dark background. It shows the execution of a Python script. The output displays the arrays for applications and seats, followed by their mean, standard deviation, demand ratio, and min-max scaled values.

```
$ python m1_numpy_basics.py
Applications : [1050 1180 1320 1200 1410]
Seats        : [800 850 900 900 950]
Mean applications : 1232.0
Std applications : 123.52
Demand ratio : [1.31 1.39 1.47 1.33 1.48]
Scaled applications: [0.  0.36 0.75 0.42 1. ]
```

Fig. 4.1 – Output of the NumPy practice program

Scaling is important because the features of the project have very different sizes: the advertisement budget runs into lakhs while the number of courses is below twelve. Scaling puts such values on a common footing for many algorithms.

4.2 Module 2 – Data Handling with pandas

Services used: Python, pandas, NumPy.

The second week was spent on pandas. A DataFrame with the seven features and the admissions column was created, saved as a CSV file, then inspected with head() and describe() and checked for missing values. The practice dataset has 200 rows and its ranges follow the typical ranges shown in the web form.

```
import numpy as np
import pandas as pd

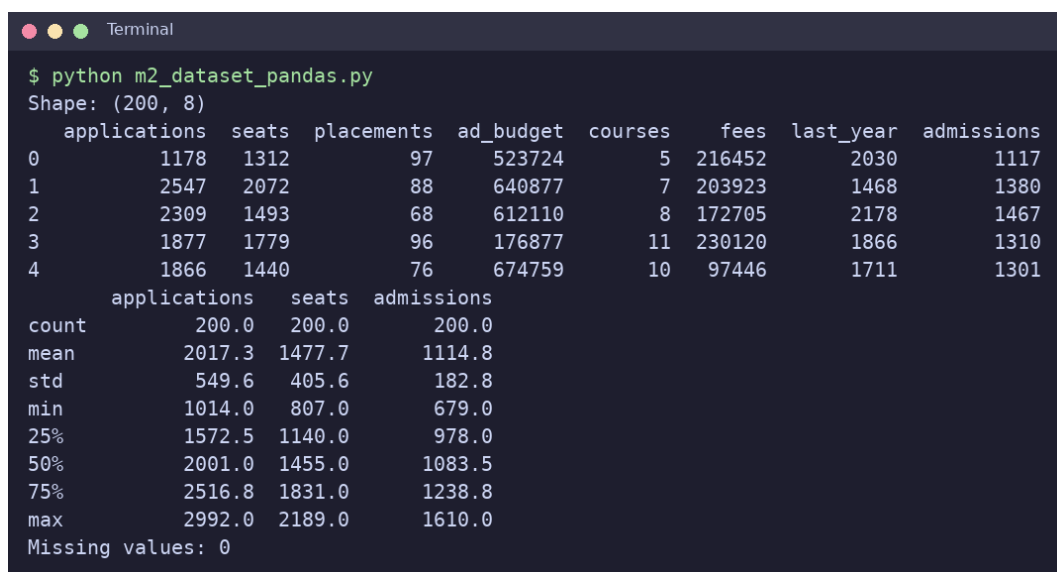
pd.set_option("display.width", 100)
pd.set_option("display.max_columns", 8)

rng = np.random.default_rng(42)
n = 200

df = pd.DataFrame({
    "applications": rng.integers(1000, 3000, n),
    "seats": rng.integers(800, 2200, n),
    "placements": rng.integers(55, 98, n),
    "ad_budget": rng.integers(80000, 800000, n),
    "courses": rng.integers(5, 12, n),
    "fees": rng.integers(75000, 250000, n),
    "last_year": rng.integers(800, 2200, n),
})

# Practice target: admissions depend on demand, seats and last year
noise = rng.normal(0, 40, n)
raw = (0.25 * df.applications + 0.30 * df.last_year
       + 3.0 * df.placements + 0.0002 * df.ad_budget
       - 0.0006 * df.fees + noise)
df["admissions"] = np.minimum(raw, df.seats).round().astype(int)

df.to_csv("practice_admissions.csv", index=False)
print("Shape:", df.shape)
print(df.head())
print(df[["applications", "seats", "admissions"]].describe().round(1))
print("Missing values:", int(df.isnull().sum().sum()))
```



```
Terminal

$ python m2_dataset_pandas.py
Shape: (200, 8)
  applications  seats  placements  ad_budget  courses  fees  last_year  admissions
0         1178   1312          97    523724         5  216452      2030         1117
1         2547   2072          88    640877         7  203923      1468         1380
2         2309   1493          68    612110         8  172705      2178         1467
3         1877   1779          96    176877        11  230120      1866         1310
4         1866   1440          76    674759        10   97446      1711         1301

  applications  seats  admissions
count      200.0   200.0        200.0
mean       2017.3  1477.7        1114.8
std         549.6   405.6         182.8
min        1014.0   807.0         679.0
25%        1572.5  1140.0         978.0
50%        2001.0  1455.0        1083.5
75%        2516.8  1831.0        1238.8
max        2992.0  2189.0        1610.0
Missing values: 0
```

Fig. 4.2 – Output of the dataset program

The output confirms that the table has 200 rows and 8 columns and that no value is missing. The

describe() summary gives the minimum, maximum, mean and quartiles, which helps to detect unreasonable values before training.

4.3 Module 3 – Model Training and Evaluation

Services used: Python, pandas, scikit-learn.

In the third week the dataset was divided into 160 training rows and 40 testing rows. A Linear Regression model and a Random Forest model were trained on the training rows and both were evaluated on the testing rows using MAE and R^2 . The trained model was then asked to predict the admissions for a new record.

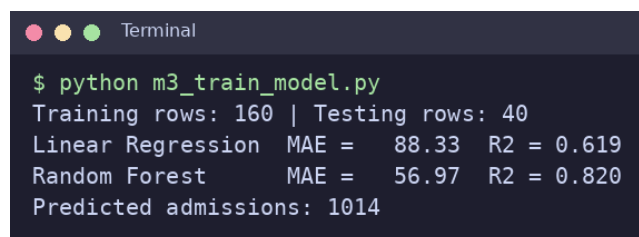
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

df = pd.read_csv("practice_admissions.csv")
X = df.drop(columns="admissions")
y = df["admissions"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print("Training rows:", len(X_train), "| Testing rows:", len(X_test))

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(f"{name:18s} MAE = {mean_absolute_error(y_test, pred):7.2f}"
          f" R2 = {r2_score(y_test, pred):.3f}")

# Predict for one new college record (same order as the columns of X)
sample = pd.DataFrame([[1800, 1200, 80, 300000, 9, 150000, 1100]],
                       columns=X.columns)
best = models["Random Forest"]
print("Predicted admissions:", int(round(best.predict(sample)[0])))
```

A terminal window with a dark background and light-colored text. The title bar shows three colored circles (red, yellow, green) and the word "Terminal". The command prompt shows the execution of a Python script: `$ python m3_train_model.py`. The output displays the number of training and testing rows, followed by performance metrics for two models: Linear Regression and Random Forest. The Random Forest model shows a lower MAE and a higher R2 score. Finally, the predicted admissions for a new record are shown as 1014.

```
Terminal
$ python m3_train_model.py
Training rows: 160 | Testing rows: 40
Linear Regression MAE = 88.33 R2 = 0.619
Random Forest MAE = 56.97 R2 = 0.820
Predicted admissions: 1014
```

Fig. 4.3 – Output of the model training program

On this practice dataset the Random Forest gave the smaller error and the higher R^2 , so it was chosen for the next steps. These figures belong to the practice dataset only and are not claimed as the accuracy of the deployed application.

4.4 Module 4 – Input Interface of the AI Admission Predictor

Services used: HTML, CSS, JavaScript.

The main project of the internship is the AI Admission Predictor. Its input page, shown in Fig. 4.4, carries the heading “AI Admission Predictor” and seven labelled fields: Applications Received, Seats Available, Placements (%), Advertisement Budget (₹), Courses Available, Annual Fees (₹) and Last Year Admissions. A small note under each field gives its typical range, which guides the user to enter realistic values. A red Predict Admissions button submits the form.

The screenshot shows a web browser window with the URL <https://admission-predictor-kaz.onrender.com>. The page title is "AI Admission Predictor". Below the title, there is a subtitle "Admission Predictor". The form consists of seven input fields, each with a label, a value, and a typical range. The values entered are: Applications Received (1200), Seats Available (900), Placements (%) (86), Advertisement Budget (₹) (400000), Courses Available (8), Annual Fees (₹) (199985), and Last Year Admissions (800). A red button labeled "Predict Admissions" is at the bottom.

Field	Value	Typical Range
Applications Received	1200	1000–3000
Seats Available	900	800–2200
Placements (%)	86	55–98
Advertisement Budget (₹)	400000	₹30,000–₹5,00,000
Courses Available	8	5–12
Annual Fees (₹)	199985	₹75,000–₹2,50,000
Last Year Admissions	800	(matches typical seats range)

Fig. 4.4 – Input form of the AI Admission Predictor (live application)

The values entered in the figure – 1200 applications, 900 seats, 86 per cent placements, an advertisement budget of ₹4,00,000, 8 courses, annual fees of ₹1,99,985 and 800 admissions last year – were used as the sample input for the results of Chapter 5. A dark colour scheme was chosen so that the labels, input boxes and the red button stand out clearly.

4.5 Module 5 – Prediction Service

Services used: Python, Flask, scikit-learn, pandas.

The prediction service receives the seven values as JSON, checks that none is missing, converts them into a one-row table in the same column order as the training data, calls the model and returns the result. The program below is a representative version of this service built on the practice dataset. It also limits the prediction to the number of seats available, because a college cannot admit more students than its sanctioned seats. It was tested with Flask’s built-in test client.

```
import pandas as pd
from flask import Flask, request, jsonify
from sklearn.ensemble import RandomForestRegressor

FEATURES = ["applications", "seats", "placements",
            "ad_budget", "courses", "fees", "last_year"]

df = pd.read_csv("practice_admissions.csv")
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(df[FEATURES], df["admissions"])

app = Flask(__name__)
```

```

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    missing = [f for f in FEATURES if f not in data]
    if missing:
        return jsonify(error="Missing fields", fields=missing), 400
    row = pd.DataFrame([[float(data[f]) for f in FEATURES]], columns=FEATURES)
    admissions = int(round(model.predict(row)[0]))
    seats = int(data["seats"])
    admissions = min(admissions, seats) # cannot exceed seats
    return jsonify(predicted_admissions=admissions,
                    seats_available=seats,
                    remaining_seats=seats - admissions)

if __name__ == "__main__":
    client = app.test_client()
    ok = {"applications": 1800, "seats": 1200, "placements": 80,
          "ad_budget": 300000, "courses": 9, "fees": 150000,
          "last_year": 1100}
    r = client.post("/predict", json=ok)
    print("Valid request  :", r.status_code, r.get_json())
    r = client.post("/predict", json={"applications": 1800})
    print("Missing fields :", r.status_code, r.get_json())

```



```

Terminal
$ python m5_flask_app.py
Valid request : 200 {'predicted_admissions': 1000, 'remaining_seats': 200,
'seats_available': 1200}
Missing fields : 400 {'error': 'Missing fields', 'fields': ['seats', 'placements',
'ad_budget', 'courses', 'fees', 'last_year']}

```

Fig. 4.5 – Output of the prediction service test

The first request returns status 200 with the predicted admissions, the seats and the remaining seats. The second request leaves out six fields and returns status 400 with the list of the missing fields, which shows that incomplete data is rejected with a clear message.

4.6 Module 6 – Result Visualisation

Services used: JavaScript, Chart.js.

After the prediction arrives, the result page shows a highlighted message “Predicted Admissions: 731” for the sample input. Below it, two charts help the user to interpret the number. The first, shown in Fig. 4.6, is a bar chart titled “Applications vs Seats vs Predicted Admissions”. It places the 1,200 applications received, the 900 seats available and the 731 predicted admissions side by side, so that the demand and the expected intake can be compared immediately.

A chart of this kind is drawn from a few lines of configuration. The following representative code defines the bar chart for the sample values.

```

const ctx = document.getElementById("barChart");

new Chart(ctx, {
    type: "bar",
    data: {
        labels: ["Applications Received", "Seats Available",
                 "Predicted Admissions"],
        datasets: [{
            data: [1200, 900, 731],

```

```

        backgroundColor: ["#3498db", "#e67e22", "#27ae60"]
      })
    },
    options: {
      plugins: { legend: { display: false } },
      scales: { y: { beginAtZero: true } }
    }
  }
});

```

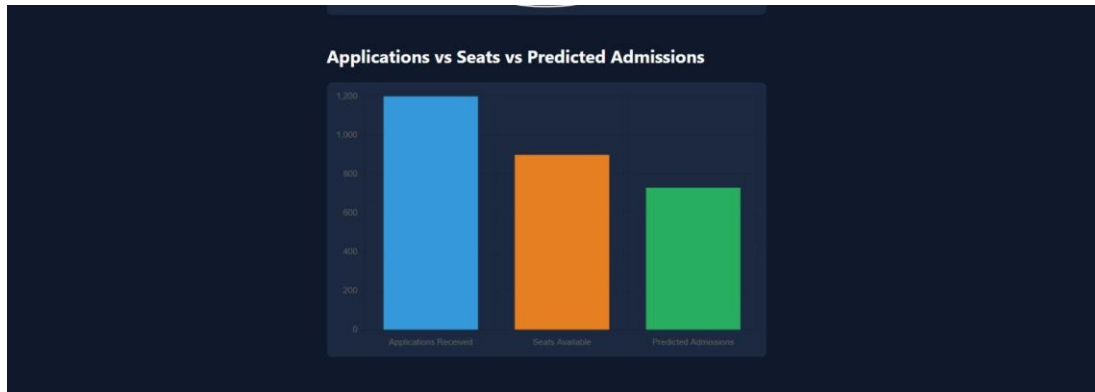


Fig. 4.6 – Bar chart of applications, seats and predicted admissions

The second chart, shown in Fig. 4.7, is a seat-utilisation doughnut. It divides the 900 seats into the filled seats predicted by the model and the remaining seats. For the sample input, 731 of 900 seats are predicted to be filled, which is 81.2 per cent, and 169 seats remain.

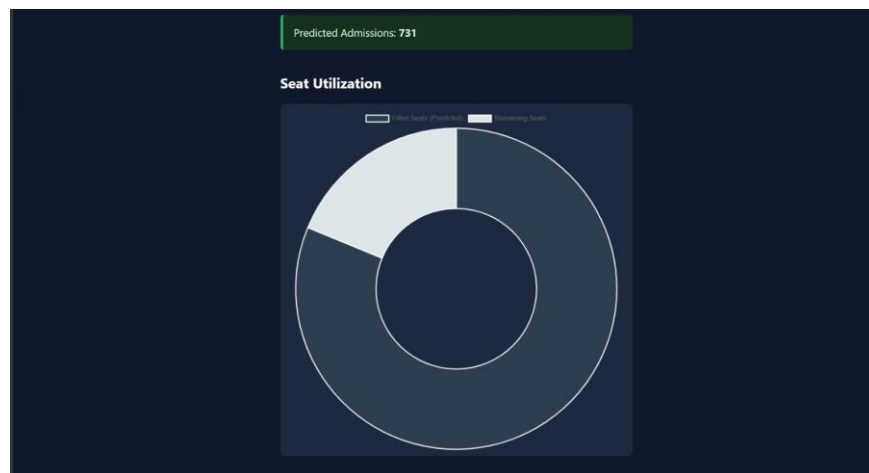


Fig. 4.7 – Predicted admissions and seat-utilisation chart

4.7 Module 7 – Deployment and Version Control

Services used: Git, GitHub, Render.

The project code was committed with Git and uploaded to a GitHub repository. The repository was then connected to Render, which installs the required libraries, starts the application and publishes it at <https://admission-predictor-kazl.onrender.com>. Whenever a new version is pushed to GitHub, the platform can rebuild the application, so the live page always reflects the latest code.

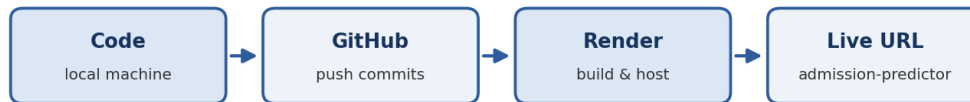


Fig. 4.8 – Deployment flow from code to live application

4.8 Module 8 – Project Poster

Services used: poster design tool.

To explain the project to people who are not technical, an infographic poster titled “College Admission Prediction AI Model” was prepared. It describes the model, lists the six factors considered, shows a sample prediction result and presents five uses of the model: admission planning, seat and capacity planning, faculty and staff planning, infrastructure development and marketing decisions. It also carries the link of the live prediction system. The figures printed on the poster are illustrative and differ from the sample values of the web application.

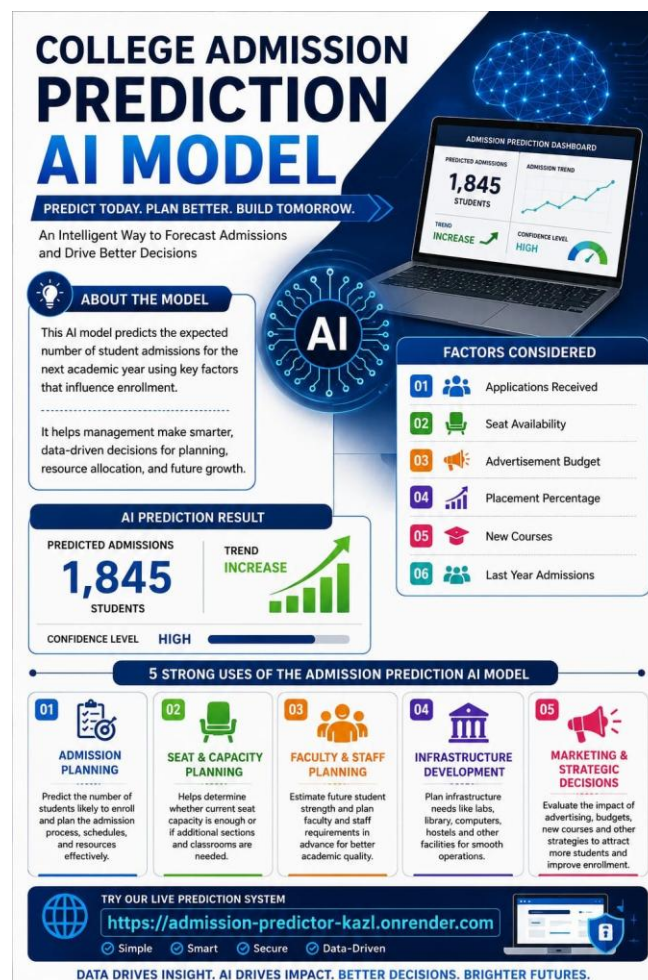


Fig. 4.9 – Project poster of the College Admission Prediction AI Model

4.9 Testing

Table 4.1 lists the checks made on the application and the practice programs.

Table 4.1 – Test cases and results

No.	Input / action	Expected result	Result
1	Open the live web address	Input form with seven fields is displayed	Passed
2	Sample values (1200, 900, 86, 4,00,000, 8, 1,99,985, 800) and Predict Admissions	Predicted admissions are displayed	Passed – 731
3	After the prediction	Bar chart of applications, seats and predicted admissions appears	Passed
4	After the prediction	Seat-utilisation doughnut appears	Passed
5	Request with missing fields (practice service)	Error message and status 400	Passed
6	Practice programs 1 – 3	Programs run without error and print results	Passed

4.10 Challenges and Solutions

Table 4.2 – Challenges faced and their solutions

Challenge	Solution
Many features with very different sizes (budget in lakhs, courses below 12)	Studied scaling with NumPy and checked the ranges with describe()
Choosing between two models	Compared Linear Regression and Random Forest on the same testing rows using MAE and R ²
Users entering unrealistic values	Showed a typical range below every field and validated the data in the back end
Prediction larger than the seats available	Limited the result to the number of seats in the prediction service
Difficult to understand a single number	Added the bar chart and the seat-utilisation doughnut
Making the project available to others	Uploaded the code to GitHub and hosted it on Render

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the outcomes of the internship, the features of the application, the skills gained and a comparison with manual estimation.

5.1 Results Obtained

The main result of the internship is a working, publicly accessible AI Admission Predictor. For the sample input used during testing, the application produced the output given in Table 5.1.

Table 5.1 – Sample input and output of the AI Admission Predictor

Item	Value
Applications Received	1200
Seats Available	900
Placements (%)	86
Advertisement Budget (₹)	4,00,000
Courses Available	8
Annual Fees (₹)	1,99,985
Last Year Admissions	800
Predicted Admissions	731
Seat utilisation (731 of 900)	81.2 %
Remaining seats (900 – 731)	169

In addition, the intern completed three practice programs, a representative prediction service, the deployment on Render, the upload of the code to GitHub and a project poster.

5.2 Features of the AI Admission Predictor

- Seven-factor input form with a typical range shown below every field.
- One-click prediction of the expected number of admissions.
- Highlighted result message for easy reading.
- Bar chart comparing applications, seats and predicted admissions.
- Seat-utilisation doughnut showing filled and remaining seats.
- Dark, high-contrast design that is easy to read.
- Live hosting on a public web address, so that no installation is needed.

5.3 Skills Gained

Table 5.2 – Skills gained during the internship

Area	Skills developed
Machine learning	Supervised learning, regression, features and labels, training and testing, MAE and R^2
Python	NumPy arrays, pandas DataFrames, scikit-learn models
Web development	Building a form-based interface and a Flask-style prediction service
Visualisation	Bar and doughnut charts to explain a prediction
Deployment	Git, GitHub and hosting on Render
Communication	Preparing a project poster and a technical report

5.4 Comparison with Manual Estimation

Table 5.3 – Manual estimation compared with the ML predictor

Aspect	Manual estimation	AI Admission Predictor
Factors considered	One or two, usually last year's figure	Seven factors together
Consistency	Varies from person to person	Same inputs give the same answer
Speed	Needs meetings and calculation	Result in a moment
Explanation	Opinion-based	Shown with a bar chart and a seat-utilisation chart
Updating	Redone by hand every year	Can be updated by retraining with new data
Availability	Depends on the people involved	Online at any time

5.5 Internship Experience

The four weeks gave the intern a clear picture of how a machine learning idea travels from a concept to a live product. Learning NumPy and pandas first made model training easy to follow, and building the interface and charts showed that a good prediction must also be communicated well.

5.6 Discussion

For the sample input, the application predicts 731 admissions against 900 seats. This is 81.2 per cent of the seats, leaving 169 seats vacant, even though 1,200 applications were received. The result is also lower than last year's 800 admissions, which shows that the output is not merely a copy of the previous year's figure.

The accuracy of such a model depends on the size and quality of the data on which it is trained. The practice programs showed the method with a generated dataset; a real deployment would require genuine historical records of a college or of several colleges. The prediction should therefore be treated as decision support and not as a guaranteed number.

5.7 Results of the Practice Programs

The model-training program of Module 3 gave the results in Table 5.4 on the practice dataset of 200 rows (160 for training and 40 for testing). They show the method of comparing models and are not the accuracy of the deployed application.

Table 5.4 – Result of the practice model comparison

Model	MAE	R ²
Linear Regression	88.33	0.619
Random Forest	56.97	0.820

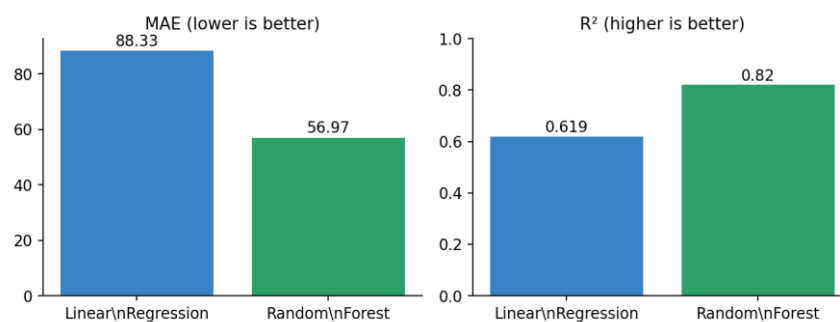


Fig. 5.1 – Comparison of the two practice models

The Random Forest has the smaller average error and explains a larger share of the variation, so it was preferred for the prediction service. A real deployment should repeat this comparison on genuine admission records before any model is trusted.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusions

The Machine Learning Programme at KanmalaiTech UK Limited gave the intern a complete, hands-on journey through the machine learning workflow. Starting from the basics of Python, NumPy and pandas, the intern learned to prepare data, train and compare regression models and measure their errors. These skills were applied in the AI Admission Predictor, a web application that converts seven admission-related factors into a predicted number of admissions and presents it with a bar chart and a seat-utilisation chart.

The project was deployed online, its code was maintained on GitHub, and a poster was prepared to explain it.

6.2 Future Work

- Train the model on real multi-year admission records from one or more colleges to improve reliability.
- Show accuracy measures such as MAE and R^2 inside the application so that users know how much to trust a prediction.
- Add a database and user login so that colleges can save and compare predictions across years.
- Add more factors, such as the cut-off marks, the location of the college and the number of competing colleges nearby.
- Retrain the model automatically each year when new admission data is added.

6.3 Skills Developed

The internship developed skills in Python, NumPy, pandas and scikit-learn, in training and evaluating regression models, in building a web interface with a prediction service, in visualising results, and in using Git, GitHub and Render.

6.4 Limitations

- The practice programs use a generated dataset and are meant to explain the method, not to measure real-world accuracy.
- The prediction depends on the seven chosen factors; other influences on admission are not considered.
- The application does not store past predictions or use a college database.

6.5 Benefits of the Internship

The internship gave practical exposure to a professional machine learning workflow, a deployed project that can be shown to employers, a better understanding of how data supports decisions, and confidence to learn further topics such as classification, deep learning and model deployment.

REFERENCES

- [1] T. M. Mitchell, Machine Learning. New York, NY, USA: McGraw-Hill, 1997.
- [2] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [4] scikit-learn developers, “scikit-learn User Guide,” [Online]. Available: https://scikit-learn.org/stable/user_guide.html
- [5] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf., 2010, pp. 56–61.
- [6] pandas development team, “pandas Documentation,” [Online]. Available: <https://pandas.pydata.org/docs/>
- [7] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.
- [8] Pallets, “Flask Documentation,” [Online]. Available: <https://flask.palletsprojects.com/>
- [9] Chart.js, “Chart.js Documentation,” [Online]. Available: <https://www.chartjs.org/docs/latest/>
- [10] Render, “Render Documentation,” [Online]. Available: <https://render.com/docs>
- [11] S. Chacon and B. Straub, Pro Git, 2nd ed. New York, NY, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [12] GitHub, “GitHub Docs,” [Online]. Available: <https://docs.github.com/>
- [13] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [14] Python Software Foundation, “Python 3 Documentation,” [Online]. Available: <https://docs.python.org/3/>
- [15] E. Matthes, Python Crash Course, 3rd ed. San Francisco, CA, USA: No Starch Press, 2023.
- [16] Google, “Machine Learning Crash Course,” [Online]. Available: <https://developers.google.com/machine-learning/crash-course>

APPENDIX

INTERNSHIP COMPLETION CERTIFICATE



CERTIFICATE OF SUCCESSFUL COMPLETION

This is to certify that **J. SAM JACOB** has successfully completed the **Machine Learning Programme** at **KanmalaiTech UK Limited**.

Throughout the programme, he demonstrated dedication, commitment, and a strong enthusiasm for learning. He actively deepened his knowledge and practical understanding of key concepts and applications in **Machine Learning**.

In recognition of his successful completion of the programme, **KanmalaiTech UK Limited** is pleased to award him this Certificate of Successful Completion.

We congratulate **J. Sam Jacob** on this achievement and wish him continued success in his future professional and personal endeavours.

Ramesh Balakrishnan
Director
KanmalaiTech UK Ltd

Date of Issue: 01-Sep-2026

A handwritten signature in black ink, appearing to read 'Ramesh B', written over a light blue rectangular background.

Signature:

Certificate of Successful Completion issued by KanmalaiTech UK Limited (dated 01-Sep-2026)